How much do the RISC-V ACT4 floating-point tests really check?

A mutation-testing study of NaN-boxing and rounding-mode coverage

Wan (Quan Pham)

October 2026 (draft)

Abstract

The RISC-V Architectural Compatibility Tests (ACT4, branch `act4` of `github.com/riscv/riscv-arch-test`) check that a processor matches the ISA specification by comparing a test "signature" with the one produced by the Sail reference model. A test suite that passes on a correct model says nothing about whether it would *fail* on a wrong one. This study measures that second property with mutation testing: bugs are injected into the Spike simulator, which then plays the role of the device under test, and we count how many of them the suite detects.

Three gaps were studied. (1) BF16 NaN-boxing when FLEN=64 (`Zfbfmin + D`): the existing suite detects **0 of 2** injected bugs; a new 6-row testplan extension (ZfbfminD, 12 generated tests) detects **2 of 2** and passes on Sail and Spike for rv32 and rv64. (2) The rounding-mode field of the exact conversions `fcvt.d.s, fcvt.d.w, fcvt.d.wu,` which ACT4 only exercises with rm=000: four mutants were built and shown to be observable with 24 hand-written cases; measuring the ACT4 kill rate for them is **pending**. (3) A static scan of the normative-rule mapping files found one dangling coverpoint reference, a class of error that no current check catches.

1. Background

1.1 How ACT4 decides pass or fail

For each unprivileged extension, a CSV testplan (`testplans/<Suite>.csv`) lists instructions as rows and coverpoints as columns. Two generators read it: `testgen` writes assembly tests (`tests/rv{32,64}i/<Suite>/*.S`) and `covergroupgen` writes SystemVerilog covergroups (`coverpoints/unpriv/<Suite>_coverage.svh`). Each test stores results in a signature region. ACT4 first runs the test on Sail to obtain the expected signature, then builds a self-checking ELF that compares the DUT's values with it and prints `RVCP-SUMMARY: TEST PASSED/FAILED.`

Functional coverage (`make coverage`) measures whether the *stimulus* hits every bin, using a Sail trace. It does not measure whether a wrong DUT would produce a different signature. That is the question mutation testing answers.

1.2 NaN-boxing

When a register is wider than the value it holds (FLEN=64 holding a 16-bit BF16), the specification requires:

- an instruction writing a narrow result sets all upper bits [63:16] to 1;
- a non-transfer instruction reading a narrow operand (e.g. `fcvt.s.bf16`) treats it as the canonical NaN (BF16 `0x7FC0`) unless [63:16] are all 1;
- transfer instructions (`fmv.x.h, fsh`) use the low 16 bits and ignore the upper bits.

The existing `Zfbfmin` suite only checks boxing of 16 bits inside 32 (coverpoints `*_S_H`). A core that boxes only up to bit 31, or checks only [31:16], passes it.

1.3 Rounding mode of exact conversions

`fcvt.d.s, fcvt.d.w` and `fcvt.d.wu` are always exact, so the rm field does not change the result, but the encoding still has an rm field: rm=101/110 are reserved and must raise an illegal-instruction exception, as must rm=111 when `frm` holds a reserved value. GNU binutils 2.42 does not accept an rm operand on these mnemonics, so ACT4 only ever emits them with rm=000. The normative rule `roundingmode_unaffected_rm_decode` in `coverpoints/norm/Ssstrict.yaml` is still marked `TODO`.

2. Method

Mutants. Each mutant is a small source change to Spike modelling a plausible RTL bug (wrong bit range, missing check, wrong datapath). Mutants are written by hand and target exactly the behaviour under study.

Kill criterion. A suite *kills* a mutant if at least one of its tests fails on the mutant Spike while all tests pass on unmodified Spike. Mutation score = killed / total.

Observability check. Before measuring a suite, each mutant is shown to be observable by a minimal test, so that a score of 0 means “the suite misses the bug”, not “the mutant is equivalent to the original”.

Environment. ACT4 `act4` at commit `ac1923b`; Sail 0.16; Spike built from source (HEAD on 2026-10-24 for ZfbfminD, commit `2c300cd` for D S4); prebuilt riscv64-unknown-elf GCC. All numbers in this draft were produced in a cloud Linux environment on 2026-10-24 and are being reproduced on the author’s machine (WSL2 Ubuntu 24.04, GCC 16.1, Verilator 5.053).

3. Study 1: ZfbfminD (BF16 NaN-boxing at FLEN=64)

3.1 Change

Two hand-written files, following the existing ZfhminD pattern for FP16:

- `testplans/Zfbfmin.csv`: an `ExtraExtensions` column and 6 rows with `ExtraExtensions = D`. The generator names the new suite `ZfbfminD`, adds `D` to `REQUIRED_EXTENSIONS`, and guards the covergroups with `` `ifdef D_SUPPORTED ``, so cores without D skip it.
- `coverpoints/norm/Zfbfmin.yaml`: maps the new covergroups to the existing normative rules (`fp_nan_boxing, fp_transfer_instrs_narrow_transfer_in/out,`

`fp_nontransfer_instrs_improper_nan_boxed_input`, and the per-op rules).

Instruction	Coverpoint	What it checks
`fcvt.bf16.s`	`cp_NaNBox_D_H`	BF16 result is boxed to 64 bits
`flh`	`cp_NaNBox_D_H`	16-bit load is boxed to 64 bits
`fmv.h.x`	`cp_NaNBox_D_H`	int-to-FP move is boxed to 64 bits
`fcvt.s.bf16`	`cp_fs1_badNB_D_H`	badly boxed input becomes canonical NaN
`fmv.x.h`	`cp_fs1_badNB_D_H`	transfer ignores the upper bits
`fsh`	`cp_fs2_badNB_D_H`	transfer ignores the upper bits

`cp_NaNBox_D_H` preloads the destination with `0x0123456789ABCDEF`, so any upper bit that the instruction fails to set shows up in the 64-bit signature. `cp_fs1_badNB_D_H` uses 14 operands whose low 16 bits are valid but whose upper bits are wrong in different places (for example `0xFFFFFFFF00000000` wrong in [31:16] only, `0x7FFFFFFFFFFF3C00` wrong in bit 63 only).

Everything else (6 covergroups, 12 tests) is generated by `EXTENSIONS=Zfbfmin make tests;` regenerating produces no diff.

3.2 Results

Config	Result
Sail rv64-max, rv32-max	12/12 PASS
Spike rv64-max, rv32-max	12/12 PASS
`prek` (format and lint hooks)	passed

Mutants:

- **M1** `fcvt.s.bf16` checks the box only in bits [31:16].
- **M2** `fmv.h.x` boxes only up to bit 31, leaving [63:32] zero.

Suite	M1	M2	Score
Existing `Zfbfmin` (6 tests)	survives	survives	**0/2**
`Zfbfmin + ZfbfminD`	killed by `ZfbfminD-fcvt.s.bf16-00`	killed by `ZfbfminD-fmv.h.x-00`	**2/2**

M1 is caught at the bin for operand `0x7FFFFFFFFFFF3C00`: bits [31:16] look correctly boxed while bit 63 does not, which is exactly the case the mutant ignores. The expected result is the canonical NaN widened to single, `0xFFFFFFFF7FC00000`. M2 is caught at the `NaNBox` bin with expected `0xFFFFFFFFFFFF3FA6`. The other four ZfbfminD tests pass, as they should, because the mutants are not in those instructions.

3.3 Remaining gap

The bad-box operands come from `bad_NaN_double_half` in `generators/testgen/src/testgen/data/edges.py`, which was designed for FP16. In BF16,

`0x7E00` is a large normal number rather than a quiet NaN; of the 14 values only `0x7FFF` is a BF16 NaN, and none is a BF16 signalling NaN. A core that skips the box check and converts a low half of `0x7F81` (BF16 sNaN) would raise the invalid flag NV, while a correct core produces the quiet canonical NaN and leaves NV clear. Adding one such operand would expose that bug through `fflags`. The same gap exists in the 32-bit `S_H` variant.

4. Study 2: rm field of exact conversions (D S4)

4.1 Mutants

Applied as one patch to Spike `2c300cd`; selected at run time with `SPIKE_MUTANT=M1..M4`, so one build serves all four.

Mutant	Models
M1	decoder treats rm as part of the opcode: only rm=000 is legal
M2	rm is never checked: reserved static rm 101/110 executes
M3	`fcvt.d.w` goes through a single-precision path when rm≠0
M4	dynamic rm (111) ignores a reserved `frm` value

4.2 Observability (measured)

24 minimal tests (`mutation/ds4-minitest/`) encode `fcvt.d.w` (input 2^24+1, which single precision cannot represent) and `fcvt.d.s` with every static rm and `frm` in {0, 5}, using `.insn` to bypass the assembler.

	none	M1	M2	M3	M4
`fcvt.d.s`, rm 1..4, 7	ok	TRAP	ok	ok	ok
`fcvt.d.s`, rm 5, 6	TRAP	TRAP	**ok**	TRAP	TRAP
`fcvt.d.w`, rm 1..4, 7 (frm 0)	ok	TRAP	ok	**WRONG**	ok
`fcvt.d.w`, rm 7, frm 5	TRAP	TRAP	**ok**	TRAP	**ok**

Every mutant differs from the reference on at least one case: 4/4 observable.

4.3 ACT4 kill rate (pending)

Suite	M1	M2	M3	M4
D, current	pending	pending	pending	pending
Ssstrict, current	pending	pending	pending	pending
D + Part A	pending	pending	pending	pending
Ssstrict + Part B	pending	pending	pending	pending

Hypothesis, to be tested: the current suites kill none of the four mutants, because they never emit these instructions with rm≠000 and Ssstrict's `cp_Ftype` sweep randomizes fmt/rs2 so it rarely lands on them.

Planned change. *Part A* (D suite): add `cp_frm` for the three instructions, with testgen emitting `.insn` encodings; expected to kill M1 and M3. *Part B* (Ssstrict): sweep rm 0..7 for the three instructions and test rm=111 with `frm` = 5, 6, 7, expecting traps where the

specification requires them; expected to kill M2 and M4. This must be rebased on open upstream PR #2619, which also edits `testplans/D.csv`. The tests can additionally run on the CVW (Wally) core through Verilator, which ACT4 already supports, to look for real RTL bugs.

5. Study 3: dangling references in the normative mapping

`coverpoints/norm/*.yaml` maps each specification rule to the coverpoints that cover it. A scan of every reference against the covergroups declared in the `.svh` files found three that do not resolve:

File	Reference	Verdict
`Vx.yaml`	`Vx16_vrgatherei16_vv_cg/cr_vl_lmul_sew16`	bug; should be `cr_vl_lmul_e16_sew16`
`Sm.yaml`	`H_mcsr_cg/cp_hcsr_access`	placeholder for future H suite
`Sstc.yaml`	`SstcH_vs_cg/cp_vs_stimecmp_accessible`	placeholder for future H suite

The scan also showed that tracker item “F S6” (ZicsrF references nonexistent coverpoints) had already been fixed by PR #2616. Nothing in the CTP build compares these references with the declarations, which is why such errors survive. A small check (warn on a missing covergroup, fail on a missing coverpoint) would prevent them.

6. Upstream status

The work targets open items in tracker issue #2644. The items were claimed on the issue on 2026-10-24. The maintainers replied that they intend to resolve the tracker items in bulk and do not currently have bandwidth to review individual pull requests. The patches, mutants and results are therefore published in this repository so they can be reused.

7. Threats to validity

- **Mutants are not RTL.** Spike mutants model the architectural effect of a bug, not its cause in hardware. They show that a class of bug is detectable, not that real cores have it.
- **Small, hand-picked mutant sets.** Six mutants, each aimed at the gap being studied. The scores show that the new tests close the specific gap; they are not a general measure of ACT4 quality. Systematic mutation operators would be a stronger test.
- **No coverage numbers.** `make coverage` needs Questa or VCS; a Verilator backend exists only as an unmerged upstream PR (#2483).
- **Single environment so far.** All numbers come from one cloud run; the reproduction on the author’s machine is in progress.

8. Future work

1. Measure the D S4 kill matrix before and after Parts A and B.
2. Run the D S4 tests on CVW via Verilator.
3. Add BF16 signalling-NaN bad-box operands (Section 3.3) and a mutant that shows the difference.
4. Write the norm-reference checker and run it in CI.
5. Automate mutant generation for FP instruction implementations in Spike.

Reproduction

See `README.md` in this repository. Every table above has a script that regenerates it, and results files mark which numbers are measured and which are pending.